

String interpolation

Document #: P3412R1
Date: 2025-01-12
Project: Programming Language C++
Audience: EWG/EWGI
Reply-to: Bengt Gustafsson
<bengt.gustafsson@beamways.com>
Victor Zverovich
<victor.zverovich@gmail.com>

Contents

1 Revision history	2
1.1 R0, Presented to EWGI in Wroclaw	2
1.2 R1, This revision	2
2 Abstract	2
3 Examples	2
4 History	3
5 Motivation	3
6 Terminology	4
7 Expression-field contents	4
7.1 Detecting where the expression ends	5
7.2 Preprocessor directives in expression-fields	6
7.3 Error handling	6
7.4 Implementation in other tools	7
8 Nested expression-fields	7
9 Encoding and raw literal prefixes	7
10 User defined suffixes	7
11 Macro expansion	8
12 Contexts where string interpolation works	8
13 String literal concatenation	9
13.1 Quoting of non-f literal contents during concatenation	10
14 Code breakage risk	10
15 Debugging feature	10
16 The <code>__FORMAT__</code> function	11
16.1 An alternative spelling	11

16.2 The standard implementation of <code>__FORMAT__</code>	11
16.3 Overloading <code><format></code>	12
17 Implementation experience	12
17.1 A stand alone implementation	12
17.2 Clang implementation	13
17.2.1 Lessons learned	13
18 Alternatives	14
18.1 Language feature	14
18.2 Reflection	14
19 Wording	14
20 Acknowledgements	14
21 References	14

1 Revision history

1.1 R0, Presented to EWGI in Wroclaw

First revision.

1.2 R1, This revision

- Remove the basic `_formatted_string` struct that R0 used to be able to unify f- and x- literals into just f-literals. This avoids relying on the P3298 and P3398 proposals.
- Remove the print overloads added by R0, instead let programmers use x-literals when printing while f-literals produce a `std::string` or `std::wstring` directly.
- Change f-literals to generate a function call to `__FORMAT__` instead of `std::make_formatted_string` or `std::format` to allow uses which do not rely on the formatting functionality of the standard library. Also a small discussion about other possible names.

2 Abstract

This proposal adds string interpolation (so called f-literals) to the C++ language. Each f-literal is transformed by the preprocessor to a token sequence constituting a call to a function tentatively named `__FORMAT__` with the expression fields extracted into a function argument list suitable for consumption by `std::format`. If `<format>` is included or if the `std` module is imported a `__FORMAT__` function which forwards to `std::format` is defined, effectively translating the f-literal to a `std::string` or `std::wstring` depending on the format string character type.

In addition this proposal adds a similar x-literal which does not enclose the extracted expressions in an argument list or wrap this in a `__FORMAT__` function call. This allows x-literals to be used with functions that have `std::format` compatible parameter lists, for instance `std::print`.

This proposal (currently the R0 revision) has been implemented in a Clang fork, which is available on Compiler Explorer. It has also been implemented as a separate program, which demonstrates the viability of this proposal for tools like syntax coloring editors.

3 Examples

Assuming a `Point` class which has a formatter we can now use string interpolation to format `Points`, and using f- or x-literals correctly there is no risk of dangling. This is in contrast with R0, where P3298 and P3398 were

required to make the below examples work as expected. Note however that the combination of f-literal and cout is no longer optimal, and just as in C++23 using std::print (with an x-literal) is preferable if performance is important.

```
Point getCenter();

std::string a = f"Center is: {getCenter()}";    // No dangling risk.

auto b = f"Center is: {getCenter()}";          // b is std::string as std::format is called

size_t len = f"Center is: {getCenter()}".size(); // Works as the f-literal is a std::string.

std::println(x"Center is: {getCenter()}");      // println("Center is: {}", getCenter()).

std::cout << f"Center is: {getCenter()}";      // Sub-optimal as a std::string is created.
```

4 History

This proposal was initiated by Hadriel Kaplan in October of 2023. Unfortunately Hadriel Kaplan never submitted his proposal officially and after some discussions and setting up an issue tracker for the proposal Hadriel Kaplan has not been possible to contact via e-mail and stopped posting on the issues in this tracker or refining his proposal.

The proposal presented here uses the same basic idea of letting the preprocessor extract the expressions out of the format string and place them as an argument list after the remaining literal. In R1 we revert back to having both f- and x-literals as described in Hadriel Kaplan's original draft.

Some parts of this proposal was taken from Hadriel Kaplan's original draft, in some instances with modifications.

Before this there was a proposal [P1819R0] which used another approach applied after preprocessing.

5 Motivation

Before this proposal:

```
int calculate(int);

std::string stringify(std::string_view prefix, int bits) {
    return std::format("{}: {}: got {} for {:#06x}", prefix, errno, calculate(bits), bits);
}

void display(std::string_view prefix, int bits) {
    std::print("{}: {}: got {} for {:#06x}", prefix, errno, calculate(bits), bits);
}
```

After this proposal:

```
int calculate(int);

std::string stringify(std::string_view prefix, int bits) {
    return f"{prefix}-{errno}: got {calculate(bits)} for {bits:#06x}";
}

void display(std::string_view prefix, int bits) {
    std::print(x"{prefix}-{errno}: got {calculate(bits)} for {bits:#06x}");
}
```

C++ f-literals are based on the same idea as python f-strings. They are wildly popular in modern python; maybe even more popular than the python `str.format()` that the C++ `std::format()` was based on.

Many other programming languages also offer string interpolation, and some use identical syntax, although there are other spellings based on a `$` or `%` prefix. ([full list](#)). As `std::format` already uses the Python syntax with `{}` it seems logical to continue on this path as there is no consensus among languages anyway.

The main benefit of f-literals is that it is far easier to see the argument usage locations, and that it is less verbose. For example in the code snippets above, in the second example it is easier to see that “`prefix`” goes before `errno`, and “`bits`” is displayed in hex. Here `errno` is used as an example of a macro that is often not known to be a macro. It would be surprising if `errno` and other macros were not allowed in f-literals, which is motivation for implementing this feature in the lexer.

IDEs and syntax highlighters can support f-literals as well, displaying the embedded expressions in a different color:

```
f"{prefix}-{errno}: got {calculate(bits)} for {bits:#06x}"
```

6 Terminology

The different parts of an f-literal have specific names, to avoid confusion. This is best illustrated by an example, see below.

```
f"The result is { get_result() :{width}.3}"
//      ~~~~~ f-string-literal ~~~~~
//      or f-literal

f"The result is { get_result() :{width}.3}"
//      ~~~~~
//      |
//      extraction-field

f"The result is { get_result() :{width}.3}"
//      ~~~~~ ~~~~~
//      |      |
//      expression-field  format-specifier

f"The result is { get_result() :{width}.3}"
//      ~~~~~
//      |
//      nested-expression-field
```

When the f-literal is passed along to the rest of the compiler a regular string literal token is formed, not containing the characters of the expression-fields. Such a string-literal token is called a *remaining-literal*.

7 Expression-field contents

The contents of an expression-field is a full *expression*. The grammar for *expression* includes the comma operator so when the expression is extracted by the preprocessor and placed after the literal each extracted expression is enclosed in a parenthesis. This prevents an extracted expression from being interpreted as multiple arguments to the function call that the f-literal results in.

Allowing a full *expression* instead of only a *assignment-expression* is needed to avoid causing errors due to commas in template argument lists, which can’t be easily differentiated from other commas by the preprocessor. This is illustrated by the following examples:

```
f"Weird, but OK: {1 < 2, 2 > 1}"

// Transformed to:
__FORMAT__("Weird, but OK: {}", (1 < 2, 2 > 1))

int values[] = {3, 7, 1, 19, 2 };
f"Reversed: {std::set<int, std::greater<>>(values, values + 5)}"

// Transformed to:
__FORMAT__("Reversed: {}", (std::set<int, std::greater<>>(values, values + 5)))
```

The main complication of allowing a full expression in an expression-field is that an expression can contain a colon, while a colon is also used to end an expression-field if there is a format-specifier.

Less problematic consequence of referring to the grammar for *expression* is that thereby nested string literals and comments using both `//` and `/* */` are allowed. Newlines are also allowed in expression-fields even if the surrounding literal is not raw. An *expression* may contain lambdas which means that there may occur other types of colons including labels and base class list introducers in lambda bodies.

It seems complicated on the standardization level to define a new *almost-expression* which has some more or less arbitrary rules limiting its contents, and it definitely increases the cognitive load on programmers to have to remember those rules. If the rules would involve escaping quotes of nested string literals with backslashes the readability is also hampered. Allowing full expressions also significantly simplifies the task if tools like `clang-tidy` would get fixup functions to change `std::format` calls to f-literals: Whatever is in the argument expressions is allowed inside the expression-field and can be copied in character by character, even including newlines and comments.

For comparison, Python has supported string interpolation for many years but in 2022 the definition of expression fields was changed to a full Python expression, including nested string literals with the same quote kind as the containing f-literal (Python allows enclosing strings in either single or double quotes, and previously nested string literals had to use the opposite quote kind compared to the enclosing f-literal). This change was proposed in [PEP-701](#) which was incorporated into Python 3.12.

7.1 Detecting where the expression ends

Detecting the end of an *expression* is easy if done while actually parsing. But calling the parser while lexing a string-literal token could be problematic, and in other tools such as syntax coloring editors don't contain a full parser, so a full parser can't be relied on.

However, it turns out that it is not very hard to implement a partial parser inside the lexer just to determine where the *expression* ends, assuming that it is possible to restart lexing from the character following the `{` that starts the expression-field. Restarting the lexing implies that nested comments, newlines, multi-character tokens etc. is handled by the normal lexing code.

Due to the fact that an expression-field must be followed by either a `}` or a `:` there are a number of rules to follow. Firstly we have to skip over matched curly braces to see which `}` is the first one after the expression, and secondly we must apply some rules to be able to discern if a colon starts a *format-specifier* or not when outside any nested curly brace pair:

- Scope resolution operators. These are double colons followed by an identifier or the `operator` keyword. If there is something else after a colon-colon token the first colon must be the start of a *format-specifier* and the second a colon used as the fill character. The standard format-specifier's fill character syntax requires a `<`, `^` or `>` character after the fill character, none of which can't start an identifier. While it is possible that a user-defined formatter allows a leading colon followed by one or more letters this seems unlikely. If such a formatter exists its users will have to change the format specifier syntax to for instance allow a space between the colons in order to be able to use f- and x-literals for their formatting.

- The colon of ternary operators must not be mistaken for the start of a *format-specifier*. This can be handled by just counting the number of `?` tokens and ignoring as many colons. An alternative, used in the Clang implementation, is to recurse to the *expression-field* handler for each `?` encountered., basically following the C++ grammar.
- The digraph `>` could be handled either by not supporting digraphs, in which case it would immediately be lexed as a colon followed by a `>` which thus means a **format-specifier** starting by a right-alignment specification. As `std::format` does not support using the digraphs `<%` and `%>` to enclose *extraction-fields* instead of braces we may assume that anyone able to type a f-literal can also type a `]` and does not have to resort to the problematic `>` digraph. An alternative, which is used in the Clang implementation, is to do a special test if an unmatched `]` token is encountered: If it was formed from the digraph sequence break it up into the separate `:` and `>` to form the expected start of the *format-specifier*. This allows using `>` as a `]` substitute as long as it is balanced within the *expression-field*.

The current implementations both use a somewhat more complex parser where nested parentheses and square brackets are also skipped over. This improves error handling by detecting mismatched parentheses in expression-fields immediately and aids in the handling of `>` digraphs. With the currently proposed syntax for reflection splicing using `[:` and `:]` to enclose an expression of type `meta::info` ignoring colons inside matched square brackets becomes mandatory.

Further into the future, if more uses of colons inside expressions are specified, the implementation of f-literal lexing may have to be updated, and such new uses of colons would have to be denied if it would mean that it is impossible to detect the end of an expression-field. Thus specifying a full *expression* as allowed in expression-fields is future proof. That is to say, the rules above need not to be written explicitly in the standard, it is enough to refer to the grammar for *expression* and the rules follow from this, including any future modifications.

7.2 Preprocessor directives in expression-fields

Preprocessor directives inside expression-fields is not allowed. It does not make much sense to allow preprocessor directives inside an expression in the first place and it may make much harm if for instance an `#else` is placed inside an expression inside a f-literal. Regardless of if the `#if` condition is true or false an unterminated string literal would result. As allowing preprocessor directives is under the control of the preprocessor implementing f-literals this limitation should be trivial to enforce.

It could be argued that some preprocessor directives or combinations should be allowed in expression fields such as `#pragma` and a complete `#if` to `#endif` combination. If there turns out to be a good use case for this the restriction on preprocessor directives could be relaxed by a later proposal.

7.3 Error handling

To handle errors inside the expression fields in a good way is somewhat challenging considering that a quote that appears inside an expression-field is the start of a nested string literal while the programmer could have missed the closing brace of an extraction-field with the intent that the quote should end the f-literal. In the simplest case this causes the *nested* string literal to be unterminated, but in cases with more string literals on the same line it may cause the inside/outside of string literals to be inverted.

```
// Here the human reader quickly detects the missing } after x, but the lexer
// will find an unterminated string literal containing a semicolon after the meters
// "identifier".
auto s = "Length: {x meters }";
```

In the Clang implementation a simple recovery mechanism is implemented by re-scanning the f-literal as a regular literal after reporting the error. This avoids follow-up errors as long as there are no string literals in the expression-fields of the f-literal. In more complex cases, just as if you miss a closing quote today, various follow up errors can be expected, especially if there are more quoted strings on the same line.

7.4 Implementation in other tools

Embedding full expressions into string literals means that both that preprocessors and tools like static analyzers and editors doing syntax coloring must be able to find the colon or right brace character that ends the expression-field. Not implementing this can have surprising results in the case of nested string literals, i.e. that the contents of the nested literal is colored as if it was not a literal while the surrounding expression-field is not colored as an expression.

```
std::string value = "Hello,";
f"Value {value + " World"}";
```

Above you can see the mis-coloring provided by the tools that produced this document.

As there may not be much of a lexer available in some tools it is a valid question how much trouble it would be to implement correct syntax coloring in those tools. It turns out that as all tokens that need to be handled are single character. So even without lexer the problem is not really hard. This has been proven by the stand alone implementation of this proposal which works on a character by character basis.

Also, many other languages include both string interpolation and syntax that allows brace pairs and ternary operators in expressions, so handling this in C++ may be just to enable this type of parsing for yet another language.

8 Nested expression-fields

Nested expression-fields inside the format-specifier of an extraction-field are always extracted regardless of if the formatter for the data type can handle this or not. While it seems odd to use the `{` character in a format-specifier for some other purpose than to start a nested expression-field it is possible for a user-defined formatter. To avoid extraction of the nested expression-field in this case you can quote a curly brace inside a *format-specifier* by doubling it as elsewhere in the f-literal. Note that no *standard* format-specifier allows braces except for dynamic width or precision, not even as fill characters.

9 Encoding and raw literal prefixes

The `f` and `x` prefixes can be combined with the `R` and `L` prefixes. Theoretically it can also be combined with with the `u`, `U` and `u8` prefixes, but as `std::format` is only available for `char` and `wchar_t` this does not currently work. Another proposal to implement formatting for other character types than `char` and `wchar_t` would be needed to address this limitation.

The order of encoding, formatting and raw prefixes is fixed so that any encoding prefix comes first, then the f-literal prefix and finally the raw literal prefix.

10 User defined suffixes

It is unclear how user defined string literal operator functions would work. As the constructor parameter of `std::format_string` must be a literal only the compile-time version of string literal operators seems feasible. But on the other hand a reasonable use case for an user-defined string literal suffix would be translation. At the cost of loosing compile time format checking it may seem feasible to allow translation via user-defined suffix as `vformat` just takes a `std::string_view` as the formatting string, but this would introduce a new way to end up with a dangling reference.

A major problem could be for the preprocessor to know whether an identifier after a f-literal is to be moved into the `formatted_string` constructor call or left behind it. Maybe this could be depending on whether there is intervening whitespace or not but at least it requires the preprocessor to check the next token before it can produce the final output for the f-literal.

Given a compile time user-defined string literal suffix `_uc` that uppercasifies the literal we get this example:

```
// What would this input translate to?
f"value: {x}"_uc;
f"value: {x}"_uc;

__FORMAT__("value: {}"_uc, (x));    // #1

// OR

__FORMAT__("value: {}", (x))_uc;    // #2
```

Given that use cases are marginal and dangling is an issue this proposal does not special treat identifiers after f-literals and a conforming implementation should produce output according to #2 regardless of if there is an intervening space or not, under the assumption that if this names a user-defined string literal suffix a compilation error will result. As a QoI improvement an implementation could produce a more specific error message detailing that f-literals can't have user-defined string literal suffixes. If we get match expressions this means that the expression `f"Value {1}"match { "Value 1" => true; _ => false; }` is expanded to `__FORMAT__("Value {}", (1))match { "Value 1" => true; _ => false; }` which should work as expected (and return true).

11 Macro expansion

Macro expansion occurs in the expression-fields just as in any expression. It must happen before the tokens in the expression-field are lifted out and placed after the remaining-literal as otherwise a macro that expands to a token sequence containing an unmatched parenthesis of some kind, a colon or a question mark could fool the rather simplistic mechanism that the preprocessor has to detect the end of an expression-field.

One issue is if the token that ends the expression field is itself part of a macro expansion. This must be an error as the remainder of that macro expansion would textually be part of the remaining-literal, which is counter-intuitive and basically requires converting the token sequence of the macro back to a string. A preprocessor implementation surely has a way to detect that a token in the lexer is part of a macro expansion and can thus produce a diagnostic in this case. In simple words the `}or :` that ends the expression-field must be part of the character string of the enclosing f-literal.

```
#define COLON :

// Ok: The COLON is part of the ternary operator expression.
f"Value: {a > b ? 1 COLON 0 :<5}";

// Bad: The macro expansion contains the end of the expression-field.
f"Value: {a > b ? 1 : 0 COLON<5}";
```

12 Contexts where string interpolation works

With the risk of stating the obvious: String interpolation only works in contexts where calling a C++ function is allowed. This excludes uses in the preprocessor such as `#include` filenames and uses in `static_assert` and the `deprecated` attribute where only a string literal is allowed. If `std::format` gains a constexpr specifier it is the intent of this proposal to allow string interpolation in places where this would allow `std::format` to be used, such as in non-type template arguments and to initiate constexpr variables. If contexts like `static_assert` and the `deprecated` attribute get the ability to handle a constant expression of character string (or `string_view`) type string interpolation should work there too.

In fact, by the transformation in the preprocessor of the f-literal to a call to `__FORMAT__` other parts of the compiler will handle the different contexts where this is or isn't allowed in different standard versions, as well as errors related to trying to format non-constant expression-fields when f-literals are placed in constexpr contexts.

It is assumed that any later proposal that makes `std::format` constexpr will also add constexpr specifiers appropriately on the standard library's implementation of `__FORMAT__` too.

13 String literal concatenation

String literal concatenation is supported in a way that moves all extracted expressions, in order, to after the concatenation of all the remaining-literals of the sequence. Only f- and x-literals are subjected to extraction if the string literal sequence contains both f-literals, x-literals and regular literals. If a concatenation sequence contains both f- and x-literals the f- literals take precedence and the combined extraction is enclosed in a `__FORMAT__` call.

Note: Letting one f or x prefix make preceding concatenated string literals f-literals is not possible as the preprocessor would have to go back and reinterpret the preceding string literals as f-literals. This may result in their end points moving, as what was thought to be the end of the literal was actually the start of a nested literal inside an expression field. This makes such a definition infeasible.

The concatenation of the remaining literals works according to the current rules, i.e. that non-raw and raw literals can be concatenated but only one encoding prefix kind may exist and it affects all literals in the sequence.

This definition allows the continued use of macros expanding to string literals which are commonly used to generate control sequences for terminals etc.

Here is a contorted example which shows different types of string literals being concatenated, some of which are expanded from macros and some of which are f- and x-literals. Note that the expansion steps shown below are for illustrative purposes only, a preprocessor/compiler is free to take other steps or just one step as long as the result on the line #2 is the same.

```
#include <format>

#define LITERAL " lucky one."
#define XLITERAL x" {name},"

const char* name = "John Doe";
L"{Hello}" XLITERAL fR"abc( you{LITERAL}})abc";    // #1

// The preprocessor first expands macros to get:
L"{Hello}" x" {name}," fR"abc( you{" lucky one.}")abc"

// Then the detection of an f-literal causes this to be transformed to:
__FORMAT__(L"{Hello}" " {"}, " R"abc( you{}})abc", (name), (" lucky one."));

// String concatenation as we know it then transforms this further to:
__FORMAT__(L"{{Hello {}}}, you{}})", (name), (" lucky one."));    // #2

// The __FORMAT__ function then calls std::format at runtime to get:
L"{Hello John Doe, you lucky one.}"    // #3
```

One of the literals at #1 above has an encoding prefix, one has x-prefix and one has f-prefix. Extraction fields are only processed in those literals that have the f- or x-prefix and that the encoding prefix extends to all the literals, while the raw prefix only applies to the immediately following literal. The expressions in all the f-literals are moved after the remaining-literal sequence to allow format to operate correctly when called from the `__FORMAT__` implementation in `<format>`

Side note: The code block above is formatted as Python which makes the f-literals colored correctly (but the initial #defines are treated as comments and the C++ comments aren't). This indicates that tools that support Python source code coloring should have limited problems with coloring C++ f-literals.

13.1 Quoting of non-f literal contents during concatenation

The example above illustrates a problem with concatenating f-literals with regular literals containing `{` or `}` characters. The programmer writing the line at #1 does not think doubling of the initial `{` character is needed as it is not in an f-literal. However, after concatenation, at #2, this quote is doubled, which is something the preprocessor has to do inside the contents of regular literals when concatenated to f-literals. Thanks to this doubling the final result again contains a single brace at #3. Note that the right brace in the output is created by the programmer doubling the brace in the source code as this is inside a f-literal. This is due to the current `std::format` rules for braces.

An alternative would be to ignore this very fringe issue and require programmers to double braces in string literals that are concatenated with f-literals, as it is such an edge case. The problem with this is that the literal could be inside a macro used with both regular and f-literals.

14 Code breakage risk

In keeping with current rules macros named as any valid prefix sequence are not expanded when the prefix sequence is directly followed by a double quote. This means that if there is a parameterless macro called `f` or `x` that can produce a valid program when placed directly before a double quote introducing string interpolation is a breaking change. The same could be said about Unicode and raw literal prefixes when these were introduced.

Due to the combinations of prefixes the macros that are no longer expanded if followed by a `"` character are:

```
f, fR, Lf, LfR, uf, uR, Uf, UfR, u8f, u8fR
x, xR, Lx, LxR, ux, uR, Ux, UxR, u8x, u8xR
```

None of these seem like a very likely candidate for a macro name, and even if such macros exist the likelihood of them being reasonable to place before a string literal without space between is low.

Depending on the contents of the macro this breakage may be silent or loud, but if the macro did something meaningful there should most often be errors flagged when the macro contents disappears and furthermore the data type will most likely change causing further errors. One macro that may cause problems is a replacement for the current `s` suffix that can be written as

```
#define f std::string() +
```

With such a macro (with one of the names listed above) some problems can be foreseen. It could be that some committee member knows of similar breakage happening when the prefixes already added after C++03 were introduced:

```
R, U, UR, u, uR, u8, u8R
```

If the committee at large does not know of such cases it seems unlikely that the new prefixes would cause many problems due to this.

15 Debugging feature

Python has a neat debugging feature which allows printing variables easier: If the expression ends in a `=` the text of the expression is considered part of the remaining-literal:

```
f"{x=}";

translates to

__FORMAT__("x={}", x);
```

The only syntactical problem with this occurs if the expression ends with `&MyClass::operator=` where the `=` would be treated as the trailing `=` unless the previous token is `operator`. It is proposed that the token sequence

`operator=` at the end of a expression-field should be treated as an error. This simple logic does not reduce programmer expressibility as you can't format a member function pointer anyway, and you can't even explicitly cast it to `void*` to be able to print the member function address.

16 The `__FORMAT__` function

The reason that the lexing of a f-literal results in a call to a function called `__FORMAT__` is to allow for code bases that don't use the standard library to still do formatting using their own facilities. The name `__FORMAT__` is tentative but the name finally selected must be something that is obscure enough to be used as an unqualified identifier without clashing.

An alternative would be to put this name in a special namespace, but then the namespace name would have to be obscure enough instead. A further alternative would be to place it in a sub-namespace of `std`. In this case we don't need an obscure name as everything is inside the `std::` namespace anyway. This has the ideological problem that a code base that doesn't use the standard library has to declare the `std` namespace itself to be able to put the implementation of the formatting function there.

16.1 An alternative spelling

It would be possible to use a special spelling for the `__FORMAT__` function to indicate that it is really special. One such spelling that would be rather logical is `operator f""()` which is consistent with how *postfix* literal operators are declared today. Note however the difference that the `f` here must be exactly that letter, we're not supporting any other prefixes as all other needs are covered by x-literals. Other than this special name the function is just a regular function, there are no restrictions on argument types other than that the first argument should be constructible from a string literal for the function to be callable.

In this variation the lexer must output an explicit call to the operator function when a f-literal is encountered:

```
f"Value {x}";

// Translates to

operator f""("Value {x}", (x));
```

Note that the lexer will have to make sure to *not* start treating the quotes after the `f` as the start of a f-literal if preceded by the operator keyword. This is needed to allow the user to write explicit calls to the operator as can be done with all other operators.

This proposal opts for a named function like `__FORMAT__` as it doesn't require any other changes to the language than to generate the function calls.

16.2 The standard implementation of `__FORMAT__`

The standard library implementation of `__FORMAT__` is located in the `<format>` header and just perfectly forwards to `std::format`. The different character types must be handled by separate overloads due to the consteval constructor of `std::basic_format_string`.

```
template<typename... Args>
std::string __FORMAT__(std::format_string<Args...> lit, Args&&... args) {
    return std::format(std::move(lit), std::forward<Args>(args)...);
}

template<typename... Args>
std::string __FORMAT__(std::wformat_string<Args...> lit, Args&&... args) {
    return std::format(std::move(lit), std::forward<Args>(args)...);
}
```

With the alternative spelling this instead becomes:

```
template<typename... Args>
std::string operator f""(std::format_string<Args...> lit, Args&&... args) {
    return std::format(std::move(lit), std::forward<Args>(args)...);
}

template<typename... Args>
std::string operator f""(std::wformat_string<Args...> lit, Args&&... args) {
    return std::format(std::move(lit), std::forward<Args>(args)...);
}
```

Regardless of which syntax is used this proposal puts the functions in the global namespace.

16.3 Overloading <format>

If a code base defines the `__FORMAT__` function exactly as the default implementation above it would not be possible to include the `<format>` header or do `import std;` as two definitions of the same function template would then be available. This can be solved by adding a constraint `requires(true)` to the user defined function to make it always be selected instead of the one provided by `<format>`. This allows `std::format` (declared in `<format>`) to be called from the user defined `__FORMAT__` function.

```
template<typename... Args> auto
__FORMAT__(std::format_string<Args...> lit, Args&&... args) requires(true) {
    return "my: " + std::format(std::move(lit), std::forward<Args>(args)...);
}

// Usage

int main()
{
    std::cout << __FORMAT__("Value: {}", 5);
}
```

Compiler Explorer: [Link](#)

17 Implementation experience

There are two implementations, both by Bengt.

17.1 A stand alone implementation

`extract_fx` is a stand alone pre-preprocessor which performs the new preprocessor tasks and produces an intermediate file that can be compiled using an unmodified C++ compiler. As this pre-preprocessor does not do macro expansion it can't support macros expanding to string literals that are to be concatenated with f-literals. All other uses of macros (including in expression-fields) are however supported by passing them on to the C++ compiler's preprocessor.

This implementation mostly works character by character but skips comments and regular string literals, avoiding translating f-literals in commented out code or inside regular literals. Inside f-literals `extract_fx` handles all the special cases noted above, except digraphs.

This implementation can be seen as a reference implementation for syntax-coloring editors and similar tools which need to know where the expression-fields are but don't need to actually do the conversion to a function call.

Implementing `extract_fx` took about 30 hours including some lexing tasks that would normally be ready-made in a tool or editor, such as comment handling.

Note: This implementation does not support x-literals but a command line switch can be used to set the name of the function to enclose the extracted expression fields in, which can be used to get the `__FORMAT__` name.

17.2 Clang implementation

There is also a Clang fork which supports this proposal [here](#), in the branch *f-literals*. This implementation is complete but lacks some error checks for such things as trying to use a f-literal as a header file name and when the end of an expression-field is inside a macro expansion. This fork does not currently support x-literals and encloses all calls in a `::std::make_formatted_string` function call.

The Clang implementation relies on recursing into the lexer from inside lexing of the f-literal itself. This turned out to be trivial in the Clang preprocessor but could pose challenges in other implementations. With this implementation strategy the handling of comments, nested string literals and macros in *expression-fields* just works, as well as appointing the correct *code location* for each token. The only thing that was problematic was that string literal concatenation is performed inside the parser in Clang rather than in the preprocessor. To solve this f-literals collect their tokenized *expression-fields* into a vector of tokens which is passed out of the preprocessor packed up with the remaining-literal as a special kind of string literal token. In the parsing of *primary-expression* the string literal is detected and new code is used to unpack the token sequence and reformat it as a `make_formatted_string` function call. This code is also responsible for the concatenation of f-literals and moving all their tokenized expression-fields to after all the remaining-literals. Writing this code was surprisingly simple.

The Clang implementation took about 50 hours, bearing in mind that the `extract_fx` implementation was fresh in mind but also that the implementer had little previous experience with “Clang hacking” and none in the preprocessor parts.

Here is an example of the two step procedure used in the Clang implementation to first create a sequence of special string-literal tokens containing the remaining-literal and token sequence for each f-literal and then handing in the parser to build the `basic_formatted_string` constructor call.

```
// Original expression:
f"Values {a} and " f"{b:.{c}}"
```



```
// The lexer passes two special string-literal tokens to the parser:
// "Values {} and " with the token sequence ,(a) and
// "{:.{c}}"
```



```
// The Parser, when doing string literal concatenation, finds that at least one
// of the literals is a f-literal and reorganizes the tokens, grabbing the stored
// strings and token sequences to form:
::std::make_formatted_string("Values {} and " "{:.{c}}", (a), (b), (c))
```



```
// This token sequence is then reinjected back into the lexer and
// ParseCastExpression is called to parse it.
```

17.2.1 Lessons learned

A point of hindsight is that with more experience with the Clang preprocessor implementation it may have been possible to avoid all changes in the parser and doing everything in the lexer. The drawback with this approach would have been that when seeing a non-f literal the lexer must continue lexing to see if more string literals follow, and if at least one f-literal exists in the sequence of string literal tokens the rewrite to a `__FORMAT__` function call can be made directly during lexing. An advantage of this is that running Clang just for preprocessing would work without additional coding, but a drawback is that for concatenated literals without any f-literal there is a small performance overhead as the literal sequence must be injected back into the preprocessor which involves

additional heap allocations. As only a small fraction of string literals involve concatenation this should not be a significant issue.

18 Alternatives

A few other approaches to get string interpolation into C++ have been proposed, which are discussed here.

18.1 Language feature

A language feature that is applied strictly after preprocessing was proposed in [P1819R0] but as the string literal is then not touched by the preprocessor it can't contain macros and nested string literals have to be escaped. This approach would still need [P3398] to avoid dangling in the simple case of assigning an auto variable to a f-literal. A bigger disadvantage seems to be that, at least according to the proposal, there is no way to implicitly convert the f-literal to a `std::string`, usage is restricted to printing and ostream insertion.

18.2 Reflection

There has been ideas floated that reflection could solve this problem. As there are no concrete proposal texts that we are aware of we can only point out a few drawbacks that seem inevitable with such an approach.

Firstly the problem with macros already being handled when reflection can see the literal is the same as with the language feature approach, as well as the need to escape nested string literals. Secondly there seems to be no inherent way that the leading `f` can be handled by reflection. A mechanism where a certain identifier can be connected to some kind of reflection mechanism would be needed. The closest approximation would be something like `std::f("...")` which is not the level of ergonomics we aim at for string interpolation.

Furthermore, when analyzing the string literal, a new mechanism to convert each extracted string to the reflection of an expression is needed. Currently it however seems that *token sequence* based code injection is more likely to be standardized than string based code injection so to support reflection based string interpolation would require additional support that can convert a `string_view` to a token sequence.

As a final remark reflection based string interpolation would be relying on compile time code execution for each f-literal which would add to compile times. The code to tokenize the string literal to find the end of expressions involves considerably more computations than the current string literal validation done by the `basic_format_string` constructor.

19 Wording

None yet.

20 Acknowledgements

Thanks to Hadriel Kaplan who initiated this effort and wrote an insightful draft proposal that was used as a starting point for this proposal and fruitful discussions in the following few months.

Thanks to Robert Kawulak and Oliver Hunt for contributing to the R1 version.

Bengt would like to thank his employer ContextVision AB for sponsoring his attendance at C++ standardization meetings.

21 References

[P1819R0] Vittorio Romeo. 2019-07-20. Interpolated Literals.
<https://wg21.link/p1819r0>